Guía paso a paso — Clase 2: Principios de las redes neuronales

Temas: modelos paramétricos · neurona de McCulloch y Pitts · el perceptrón · regla de aprendizaje · práctica en Python
Duración: ~2 h 30 min
Material: diapositivas `Clase2_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
Requisito: haber terminado la instalación de la Clase 1 (GUIA_PASO_A_PASO_CLASE1.md, Parte A)
Lecturas base (libro del Dr. Irving Vasquez):

- Modelo de McCulloch y Pitts
- El perceptrón
- Regla de aprendizaje del perceptrón

Objetivos de la clase

Al terminar, cada alumno podrá:

1. Calcular a mano la salida de una neurona de McCulloch y Pitts y de un perceptrón.
2. Aplicar un paso de la regla de aprendizaje de Rosenblatt.
3. Explicar por qué un perceptrón no puede aprender XOR.
4. Programar ambas neuronas en Python.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora, paso a paso.
- **Parte D:** problemas frecuentes.

Agenda

Tiempo	Tema	Diapositivas
0:00	Presentación, objetivos y repaso de la Clase 1 (5 min)	1–4
0:05	B1. Modelos paramétricos (10 min)	5
0:15	B2. Neurona de McCulloch y Pitts (25 min)	6–9
0:40	C1. Práctica: McCulloch y Pitts (20 min)	10
1:00	Descanso (10 min)	11
1:10	B3. El perceptrón (20 min)	12–15
1:30	B4. Regla de aprendizaje (20 min)	16–18

Tiempo	Tema	Diapositivas
1:50	C2–C5. Práctica: perceptrón (30 min)	19–20
2:20	Cierre, quiz y tarea (10 min)	21

PARTE A — Antes de clase (15 min)

A.1 Actualizar el repositorio

Si ya clonaste el repo en la Clase 1, descarga los cambios nuevos. En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

Verificación: debe decir `Already up to date.` o listar archivos actualizados.

Si **no** lo tenías clonado, sigue la Clase 1, Parte A, Pasos 4 a 6.

A.2 Activar el ambiente y comprobar que todo funciona

En la misma terminal:

```
conda activate nni
```

```
python docs/practica_02.py
```

Verificación: al final debe imprimir `Todas las comprobaciones pasaron.` Si las tildes salen como `�`, ejecuta primero `chcp 65001`; es solo visual.

A.3 Checklist

- [ ] `python docs/practica_02.py` funciona.
- [ ] VS Code abre `notebooks/1_01_mcculloch_pitts.ipynb` con el kernel **nni**.
- [ ] Plan B probado: el mismo notebook abre en Google Colab.
- [ ] En el pizarrón, un plano cartesiano con los 4 puntos (0,0), (0,1), (1,0), (1,1). Lo usarás tres veces.
- [ ] Diapositivas abiertas (`Clase2_Diapositivas.pdf` o `.pptx`).

PARTE B — Teoría desde cero

Repaso de la Clase 1 (5 min) — diapositivas 1–4

Pregunta a la clase:

- "¿Qué es aprendizaje supervisado?" → aprender de pares (entrada x, etiqueta y).
- "¿Qué es generalizar?" → acertar en ejemplos nunca vistos.

Hoy construimos la **primera pieza** de una red neuronal: la neurona artificial.

B1. Modelos paramétricos (10 min) — diapositiva 5

Idea en una frase: un modelo paramétrico es una fórmula con forma fija, cuyo comportamiento cambia según unos números llamados **parámetros**.

$$\hat{y} = \Phi_\theta(X)$$

Símbolo	Se lee	Significa
X	"equis"	La entrada (los datos que le damos)
θ	"theta"	Los parámetros (las perillas que se ajustan)
Φ	"fi"	La fórmula o modelo
ŷ	"y gorro"	La predicción del modelo
y	"y"	La respuesta correcta (etiqueta)

Ejemplo en el pizarrón: la recta ŷ = w·x + b.

- Dibuja tres rectas: (w=1, b=0), (w=2, b=1), (w=−1, b=3).
- La **forma** es siempre una recta; los **parámetros** w (inclinación) y b (desplazamiento) cambian cuál recta es.

Idea central de todo el curso:

> **Aprender = encontrar los parámetros θ que hacen que ŷ se parezca a y.**

Formalmente: |y − ŷ| < ε, con ε (épsilon) muy cercano a cero.

En redes neuronales, los parámetros se llaman **pesos (W)** y **sesgo (b)**.

B2. La neurona de McCulloch y Pitts (25 min) — diapositivas 6–9

Historia (2 min)

- **1943.** Warren McCulloch (neurocientífico) y Walter Pitts (lógico matemático).
- Fue la **primera neurona artificial**: una simplificación muy abstracta de una neurona biológica.
- Propusieron que conectando muchas de estas neuronas se puede calcular **cualquier función lógica**. Fue el primer puente entre el cerebro y la computación.

Cómo funciona (8 min)

La neurona se llama **C** y tiene tres elementos:

Elemento	Símbolo	Valores	Papel
Entradas **excitatorias**	$E = \{e_1, \ldots, e_n\}$	0 o 1	Empujan a la neurona a activarse
Entradas **inhibitorias**	$I = \{\iota_1, \ldots, \iota_m\}$	0 o 1	Si alguna vale 1, apagan la neurona
Umbral	u	0, 1, 2, …	Cuántas excitatorias encendidas hacen falta

La salida es $\hat{y} = C(E, I, u)$, y solo puede valer 0 (apagada) o 1 (activada).

Reglas (escríbelas en el pizarrón en este orden):

1. Si **alguna** entrada inhibitoria vale 1 → **$\hat{y} = 0$**.
2. Si no, suma las excitatorias: si $\Sigma\, e_i \geq u$ → **$\hat{y} = 1$**.
3. En cualquier otro caso → **$\hat{y} = 0$**.

Analogía: un portero de discoteca.

- Las inhibitorias son una "lista negra": si estás en ella, no entras, punto.
- Si no, entras solo si traes al menos *u* invitaciones (excitatorias).

Cómo se dibuja (notación de Minsky):

- Medio círculo = la neurona, con el umbral escrito dentro.
- Flecha = entrada excitatoria.
- Línea terminada en circulito = entrada inhibitoria.
- Línea de salida = el axón.

Ejemplos resueltos (10 min)

Hazlos en el pizarrón **preguntando a la clase cada fila**.

Ejemplo 1 — compuerta NOT: $C(E = \emptyset, I = \{\iota_1\}, u = 0)$. E vacío significa que no hay excitatorias, así que la suma es 0.

ι_1	$\hat{y}$	¿Por qué?
0	1	No hay inhibición y la suma $0 \geq u = 0$
1	0	La inhibitoria está activa

Resultado: la salida es lo contrario de la entrada. Es un **NOT**.

Ejemplo 2: $C(E = \{e_1\}, I = \{\iota_1\}, u = 1)$

e_1	ι_1	$\hat{y}$
0	0	0
0	1	0
1	0	**1**
1	1	0

Solo se activa con $e_1 = 1$ y $\iota_1 = 0$: implementa **"e_1 Y NO ι_1"**.

Ejemplo 3 — AND y OR con dos excitatorias:

e_1	e_2	Suma	AND (u = 2)	OR (u = 1)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	2	1	1

Regla general con n excitatorias:

Compuerta	Configuración
AND	u = n (todas encendidas)
OR	u = 1 (al menos una)
"al menos k de n"	u = k
NOT	E = ∅, una inhibitoria, u = 0

Limitaciones (5 min)

Pregunta a la clase primero: "¿Qué le falta a esta neurona?"

1. **Sin pesos:** todas las entradas valen igual. No puede decir "esta entrada importa más".
2. **Solo binaria:** no acepta valores como 3.7 metros o 25 grados.
3. **No aprende:** el umbral lo fijamos nosotros a mano. ← la más importante.

Esto motiva el perceptrón.

Ejercicios en papel (5 min)

1. ¿Qué umbral necesita una neurona con 3 excitatorias para activarse con "al menos 2 de 3"?
2. Escribe la tabla de C(E = {e_1, e_2}, I = ∅, u = 2).
3. Con C(E = {e_1, e_2}, I = {ι_1}, u = 1), ¿qué sale cuando $\iota_1 = 1$?

Respuestas:

1. u = 2.
2. 0, 0, 0, 1 (es AND).
3. Siempre 0.

B3. El perceptrón (20 min) — diapositivas 12–15

Historia (2 min)

- **1958.** Frank Rosenblatt.
- El primer perceptrón fue una **máquina física**: potenciómetros, motores y una cuadrícula de fotoceldas como "ojos". Reconocía figuras simples y **ajustaba sus pesos cuando se equivocaba**.

Analogía biológica

Neurona biológica	Perceptrón
Dendritas (reciben señales)	Entradas X
Núcleo (procesa)	Suma ponderada
Axón (envía la señal)	Salida ŷ

Qué mejora respecto a McCulloch y Pitts

	McCulloch y Pitts	Perceptrón
Entradas	Solo 0 o 1	Cualquier número real
Importancia	Todas iguales	Cada una con su **peso** w_i
Desplazamiento	Umbral u fijo	**Sesgo** b ajustable
Aprende	No	**Sí**

El modelo en dos pasos (8 min)

Paso 1 — combinación lineal: multiplicar cada entrada por su peso, sumar todo y sumar el sesgo.

$$h = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \sum_{i=1}^{n} w_i x_i + b$$

Paso 2 — función de activación escalón: decide si se activa.

$$\hat{y} = f(h) = \begin{cases} 1 & \text{si } h \geq 0 \\ 0 & \text{si } h < 0 \end{cases}$$

Analogía: decidir si salir a correr.

- x_1 = ¿hace sol? (1/0), x_2 = ¿estoy cansado? (1/0).
- Pesos: $w_1 = 2$ (el sol me anima mucho), $w_2 = -3$ (el cansancio me frena más).
- Sesgo $b = -1$ (soy algo perezoso).
- Sol y descansado: $h = 2 \cdot 1 - 3 \cdot 0 - 1 = 1 \geq 0$ → **salgo** ($\hat{y} = 1$).
- Sol y cansado: $h = 2 - 3 - 1 = -2 < 0$ → **no salgo** ($\hat{y} = 0$).

Notas:

- Una variante es la función **signo**, que devuelve −1 en lugar de 0.
- **Truco del sesgo:** agregar una entrada extra $x_0 = 1$ cuyo peso es b. Así $h = W \cdot X$ y el sesgo es "un peso más". Los notebooks lo hacen con `np.insert`.

Ejemplo resuelto del libro (5 min)

W = [0.7, −0.3, 0.4], b = 0, X = [8, 5, 6]

1. $h = 0.7 \cdot 8 + (-0.3) \cdot 5 + 0.4 \cdot 6$
2. $h = 5.6 - 1.5 + 2.4 =$ **6.5**
3. $\hat{y} = f(6.5) =$ **1**, porque $6.5 \geq 0$.

Interpretación geométrica (5 min) — usa el plano del pizarrón

Con dos entradas, la ecuación $w_1x_1 + w_2x_2 + b = 0$ es una **recta** (la **frontera de decisión**):

- de un lado, $h \geq 0 \rightarrow$ clase 1;
- del otro lado, $h < 0 \rightarrow$ clase 0.

Con n entradas, la frontera es un **hiperplano**.

AND: $w_1 = 1$, $w_2 = 1$, $b = -1.5 \rightarrow$ la recta $x_1 + x_2 = 1.5$. Solo el punto (1,1) queda arriba. Compruébalo:

x_1	x_2	$h = x_1 + x_2 - 1.5$	$\hat{y}$
0	0	−1.5	0
0	1	−0.5	0
1	0	−0.5	0
1	1	0.5	1

OR: $w_1 = 1$, $w_2 = 1$, $b = -0.5$.

XOR — actividad (2 min): XOR vale 1 en (0,1) y (1,0), y 0 en (0,0) y (1,1). "Intenten trazar UNA recta que separe los unos de los ceros." → **Es imposible.** Decimos que XOR **no es linealmente separable**.

B4. Regla de aprendizaje de Rosenblatt (20 min) — diapositivas 16–18

El problema

¿Cómo encuentra el perceptrón **solo** los pesos correctos a partir de ejemplos (x, y)?

El algoritmo

1. **Inicializar:** pesos y sesgo con valores aleatorios pequeños, por ejemplo entre −1 y 1, o en cero.
2. **Para cada ejemplo** (X, y):
    1. Calcular la predicción $\hat{y}$.
    2. Calcular el error: $e = y - \hat{y}$.
    3. Actualizar los pesos y el sesgo:

$$W \leftarrow W + \eta\,(y - \hat{y})\,X \qquad b \leftarrow b + \eta\,(y - \hat{y})$$

3. **Repetir** pasadas por todo el dataset hasta que no haya errores, o hasta un máximo de pasadas.

Vocabulario

Término	Significado
η (eta)	**Tasa de aprendizaje:** qué tan grande es cada corrección
$e = y - \hat{y}$	**Error:** indica la dirección de la corrección
Época	Una pasada completa por todos los ejemplos
Convergencia	Llegar a cero errores

Por qué funciona (intuición)

Caso	Error	Qué hace la regla	Efecto
Acertó	0	Nada	Si funciona, no lo toques
Dijo 0, debía decir 1	+1	Suma η·X a los pesos	h sube la próxima vez
Dijo 1, debía decir 0	−1	Resta η·X a los pesos	h baja la próxima vez

Teorema de convergencia (Novikoff, 1962): si los datos son **linealmente separables**, la regla encuentra una solución en un número **finito** de pasos. Con XOR nunca converge.

Ejemplo resuelto del libro (8 min) — hazlo paso a paso en el pizarrón

Datos: $X = [3, 5, 8]$, $y = 0$, $\eta = 0.5$, $W^0 = [0.3, -0.4, 0.5]$, $b^0 = 0.2$

1. **Predicción:** $h = 0.3 \cdot 3 + (-0.4) \cdot 5 + 0.5 \cdot 8 + 0.2 = 0.9 - 2.0 + 4.0 + 0.2 = 3.1 \rightarrow \hat{y} = 1$.
2. **Error:** $e = 0 - 1 = -1$. Se equivocó, hay que corregir.
3. **Nuevos pesos:** $W^1 = [0.3, -0.4, 0.5] + 0.5 \cdot (-1) \cdot [3, 5, 8] = [0.3 - 1.5, -0.4 - 2.5, 0.5 - 4.0] =$ **$[-1.2, -2.9, -3.5]$**.
4. **Nuevo sesgo:** $b^1 = 0.2 + 0.5 \cdot (-1) =$ **-0.3**.
5. **Comprobar:** $h = -3.6 - 14.5 - 28 - 0.3 = -46.4 < 0 \rightarrow \hat{y} = 0 = y$ ✓.

Ejercicios en papel (10 min)

A. $w_1 = 0.5$, $w_2 = -0.3$, $b = 0.4$, función escalón, $X = [1, 2]$, $y = 0$, $\eta = 0.1$.

- a) ¿Cuánto vale $\hat{y}$?
- b) ¿Cuál es el error?
- c) ¿Cuánto cambian los pesos y el sesgo?
- d) ¿Cuáles son los nuevos valores?

B. Se ejecutan 50 épocas sobre un dataset de 27 ejemplos. ¿Cuántas veces en total se presenta un ejemplo? ¿Cuántas de esas son repeticiones?

C. Encuentra a mano pesos y sesgo para NAND, que vale 0 solo en (1,1).

Respuestas:

- **A:**
  - a) $h = 0.5 - 0.6 + 0.4 = 0.3 \geq 0 \rightarrow \hat{y} = 1$.
  - b) $e = -1$.
  - c) $\Delta W = [-0.1, -0.2]$, $\Delta b = -0.1$.
  - d) $W = [0.4, -0.5]$, $b = 0.3$.
- **B:** $50 \times 27 = 1350$ presentaciones; $1350 - 27 = 1323$ repeticiones.
- **C:** $w = [-1, -1]$, $b = 1.5$ (es AND con los signos invertidos).

PARTE C — Práctica en la computadora

Preparación (todos):

1. Abre VS Code → **File** → **Open Folder** → `C:\Users\TuUsuario\projects\curso-redes-neuronales`.
2. Para cada notebook, revisa arriba a la derecha que el kernel sea **nni** (Select Kernel → Python Environments → nni).
3. En los notebooks, las partes a completar están marcadas con `# TODO`. Ejecuta las celdas **en orden**, de arriba abajo, con **Shift + Enter**.

C1. Notebook `notebooks/1_01_mcculloch_pitts.ipynb` (20 min, 3 puntos) — diapositiva 10

Qué hay que hacer:

1. **TODO 1 (2 pts):** completar la función `neuronaMyP(E, I, u)` **sin usar funciones de NumPy**, solo `for`, `if` y sumas.
2. Ejecutar la celda de prueba: con `E = [1]`, `I = [0]`, `u = 1` la salida debe ser **1**.
3. **TODO 2 (1 pt):** programa que reciba E, I y u cualesquiera y devuelva la salida.

Pistas para los alumnos (no la solución):

- Traduce las 3 reglas de B2 a código, **en el mismo orden**.
- Primero recorre `I`: si encuentras un 1, devuelve 0 de inmediato.
- Luego suma los elementos de `E` con un `for`.
- Compara la suma con `u`.

Cómo verificar (que lo prueben ellos): las tablas de los Ejemplos 1, 2 y 3 de B2 deben salir iguales.

- `neuronaMyP([], [0], 0)` → 1
- `neuronaMyP([], [1], 0)` → 0
- `neuronaMyP([1, 1], [], 2)` → 1
- `neuronaMyP([1, 0], [], 2)` → 0

Error típico: con E vacío la suma es 0, y con u = 0 la neurona **sí** se activa.

C2. Demo del docente: `docs/practica_02.py` (5 min) — diapositiva 19

Proyecta y ejecuta en Anaconda PowerShell (con `(nni)` activo, dentro del repo):

```
python docs/practica_02.py
```

Qué mostrar en la salida:

- McCulloch-Pitts reproduce AND, OR y NOT.
- El perceptrón **aprende** AND y OR en 4 épocas.
- Con XOR, `convergió=False` después de 100 épocas: la demostración en vivo de que no es linealmente separable.

Abre el archivo y señala las tres funciones: `predecir` (los pasos 1 y 2 de B3), `entrenar` (la regla de B4) y `escalon`.

C3. Notebook `notebooks/1_02_perceptron_AND.ipynb` (5 min, práctica) — diapositiva 20

1. Cambia `weight1`, `weight2` y `bias` **a mano**.
2. Ejecuta la celda de comprobación.
3. Repite hasta que las 4 filas digan `Yes`.

Pista: usa la interpretación geométrica de B3.

C4. Notebook `notebooks/1_03_1_perceptron_simple.ipynb` (10 min, 3 puntos)

1. **TODO (1 pt):** `combinacion_lineal(W, X, b)` **sin** `np.dot`: un `for` que acumule `w * x` y al final sume `b`.
2. **TODO (1 pt):** `escalon(h)`: debe devolver 1 cuando h ≥ 0 y 0 si no. Debe funcionar con un número o con un arreglo. Pista: `(h >= 0)` convertido a entero.
3. Ejecuta la pasada frontal de ejemplo: con `inputs = [0.7, -0.3]`, `weights = [0.1, 0.8]`, `bias = -0.1` → h = −0.27 → salida **0**.
4. **TODO (1 pt):** encontrar por prueba y error pesos para **OR**.

C5. Notebook `notebooks/1_03_2_regla_de_aprendizaje.ipynb` (tarea o tiempo extra, 4 puntos)

1. **Inicialización (1 pt):** pesos con `np.random.normal(loc=0, scale=1, size=(2,))` y sesgo en 0.
2. **Pérdida SAE (1 pt):** suma de los valores absolutos de `y_true - y_pred`. Mide cuántos ejemplos falla.
3. **Regla de aprendizaje (1 pt):** dentro del `for`, aplicar las fórmulas de B4 a `perceptron.W` y `perceptron.b`.
4. **Hiperparámetros (2 pts, según el notebook):** con 3 épocas suele quedar incompleto. Probar `n_epocas` entre 20 y 50 y `tasa_aprendizaje` entre 0.01 y 0.5. Observar:
   - la gráfica de pérdida, que debe bajar;
   - la frontera de decisión, que debe separar la mayoría de puntos rojos y azules.

> Las soluciones de los `TODO` están en la clave del docente, que queda solo en la computadora del profesor y no está en el repositorio.

Cierre (10 min) — diapositiva 21

Resumen en tres líneas:

1. Neurona artificial = suma ponderada + función de activación.
2. El perceptrón aprende corrigiendo sus pesos con el error: $W \leftarrow W + \eta(y - \hat{y})X$.
3. Solo puede separar clases con una recta: problemas **linealmente separables**.

Cliffhanger: "XOR no se puede con una sola neurona. Minsky y Papert lo demostraron en 1969 en su libro *Perceptrons*, y la investigación casi se detuvo por años. ¿La solución? Conectar neuronas en **capas**. Lo veremos en las próximas clases."

Quiz de salida (en papel, 3 min):

Pregunta	Respuesta
¿Qué tiene el perceptrón que no tiene McCulloch-Pitts?	Pesos, entradas reales y aprendizaje
¿Qué hace una entrada inhibitoria activa?	Fuerza la salida a 0
Si ŷ = y, ¿cuánto cambian los pesos?	Nada (el error es 0)
¿Qué es una época?	Una pasada completa por el dataset
¿Por qué el perceptrón no aprende XOR?	No es linealmente separable

Tarea:

1. Terminar los notebooks `1_01`, `1_03_1` y `1_03_2` (10 puntos en total).
2. En `docs/practica_02.py`:
   - cambia `eta` a 0.01 y a 1: ¿cambia el número de épocas para AND?
   - dibuja en papel la recta aprendida para OR junto a los 4 puntos.
3. Leer del libro: Neurona artificial moderna, preparación de la Clase 3.

Rúbrica:

Notebook	Puntos
1_01 McCulloch-Pitts	3
1_03_1 Perceptrón simple	3
1_03_2 Regla de aprendizaje	4
1_02 Perceptrón AND	Práctica, sin puntos

PARTE D — Problemas frecuentes

Síntoma	Causa	Solución
`NameError: name 'np' is not defined`	No se ejecutó la celda de `import`	Ejecuta las celdas desde la primera, en orden
`ModuleNotFoundError: No module named 'nni'` (en `1_03_2`)	El notebook no se abrió desde la carpeta del repo	Abre en VS Code la carpeta `curso-redes-neuronales` completa; en Colab, clona el repo y haz `%cd curso-redes-neuronales/notebooks`
`ModuleNotFoundError: No module named 'sklearn'`	Falta scikit-learn	Con `(nni)` activo: `pip install scikit-learn`
La función devuelve `None`	Falta el `return`	Revisa que todas las ramas terminen en `return`
El perceptrón no mejora	Muy pocas épocas o η muy pequeña	Sube `n_epocas` a 20–50

Síntoma	Causa	Solución
La pérdida sube y baja sin parar	Los datos no son separables, o η es muy grande	Baja η; acepta "mayoría bien clasificada"
`ValueError: shapes not aligned`	W y X de distinto tamaño	Imprime `W.shape` y `X.shape`; deben coincidir
El kernel no aparece como `nni`	Falta `ipykernel`	`pip install ipykernel` y reinicia VS Code
Tildes como `?` en la terminal	Codificación de Windows	`chcp 65001`